Programming ARM Boards

Unit II

Dr. Ayush Kumar Agrawal

SoCS, UPES

Introduction to ARM and STM32

Lecture Objectives

By the end of this lecture, students will be able to:

- Understand why ARM dominates modern embedded systems
- Differentiate between microcontrollers and SoCs
- Explain ARM licensing and Cortex families
- Identify STM32 architecture and ecosystem
- Understand how programming ARM boards differs from 8051 or Arduino

Why Study ARM-Based Systems?

- Modern embedded systems are no longer simple control units
- They require:
  - High computational capability
  - Real-time response
  - Low power consumption
  - Integration with communication protocols
- ARM processors provide the best balance between performance and power
- Over 90% of embedded processors today are ARM-based

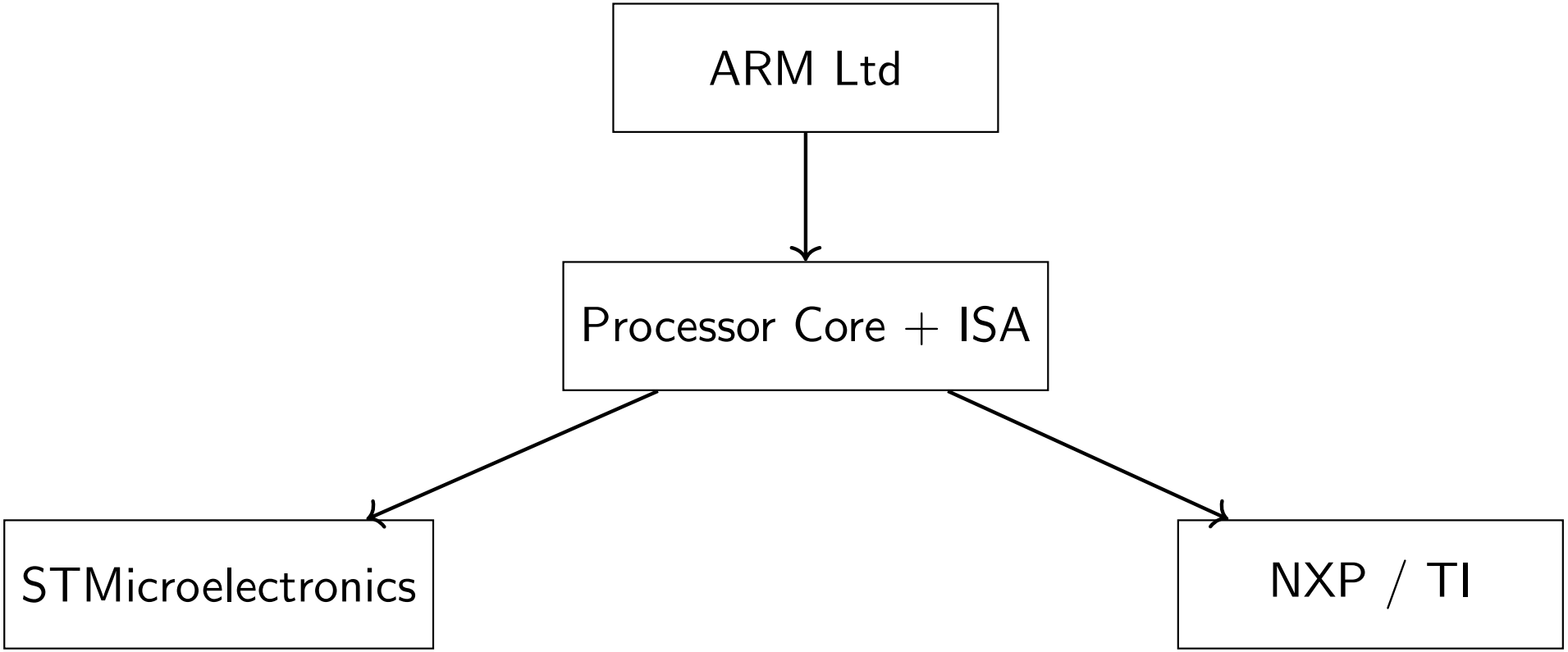
Evolution of Embedded Processors

- Early systems used:
  - 8-bit microcontrollers (8051, PIC)
  - Limited memory and peripherals
- Growth of IoT, robotics, and automation demanded:
  - 32-bit processing
  - Rich peripherals
  - Standardized architecture
- ARM emerged as the industry standard

What is ARM?

- ARM stands for Advanced RISC Machines
- It is not a chip manufacturer
- ARM designs:
  - Processor cores
  - Instruction Set Architecture (ISA)
- Semiconductor companies license ARM designs and build chips

ARM Licensing Model



ARM licenses designs; vendors integrate peripherals and memory

Microcontroller vs System-on-Chip

Microcontroller

A microcontroller integrates CPU, memory, and peripherals on a single chip for control-oriented tasks.

System-on-Chip

An SoC integrates processor, memory, buses, peripherals, and debug logic on a single silicon die.

- STM32 is a microcontroller-class SoC
- Everything required for execution is on-chip

ARM Cortex Processor Families

- Cortex-A:
  - Application processors
  - Runs Linux and Android
- Cortex-R:
  - Real-time processors
  - Automotive and safety-critical systems
- Cortex-M:
  - Microcontroller processors
  - Deterministic and low power

Why Cortex-M for Embedded Systems?

- Optimized for real-time applications
- Low interrupt latency
- Simple and efficient instruction set
- Integrated NVIC (Nested Vectored Interrupt Controller)
- Ideal for GPIO, timers, ADC, and communication protocols

Cortex-M Variants in STM32

- Cortex-M0/M0+: Ultra-low power, basic control
- Cortex-M3: Balanced performance, no FPU
- Cortex-M4: DSP instructions and FPU
- Cortex-M7: High-speed processing

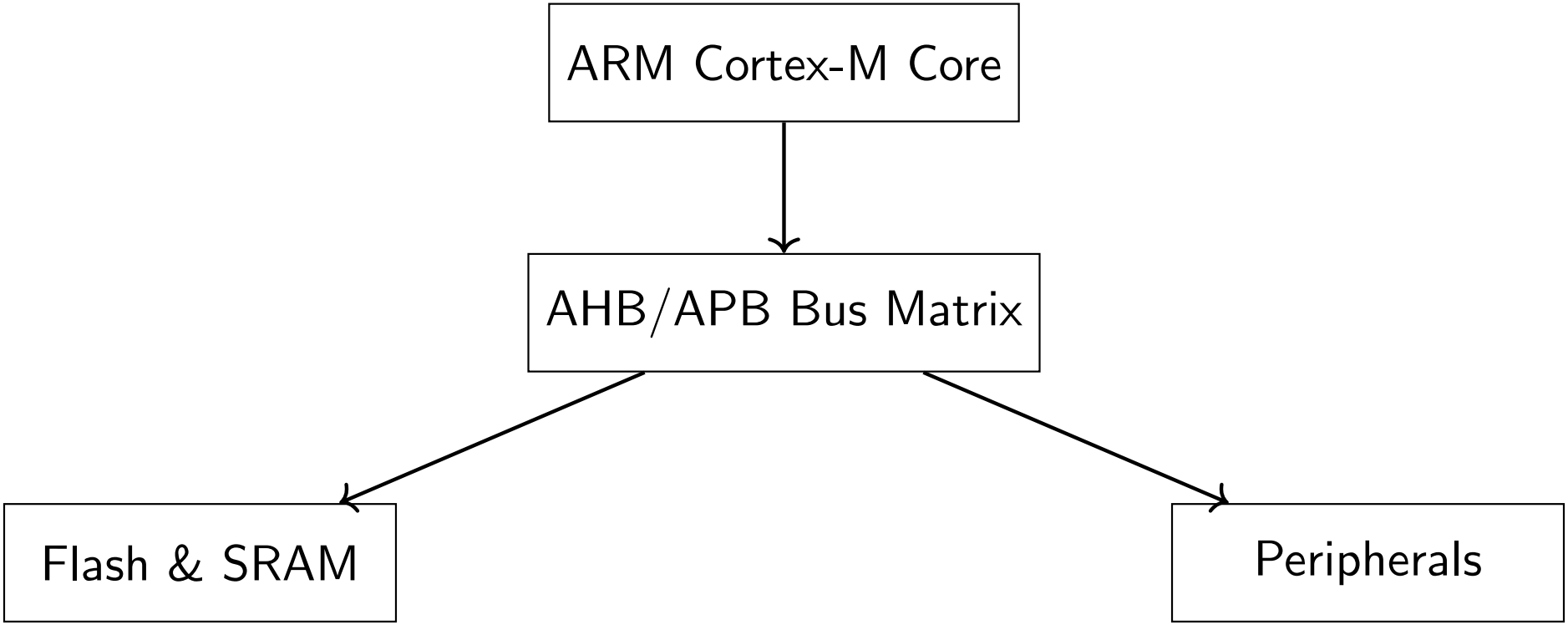
Course Focus

We mainly focus on Cortex-M3 and Cortex-M4 based STM32 boards

What is STM32?

- STM32 is a family of ARM Cortex-based microcontrollers
- Designed and manufactured by STMicroelectronics
- Widely used in:
  - Industrial automation
  - Robotics
  - IoT devices
  - Medical electronics

STM32 High-Level Architecture



STM32 Memory-Mapped Architecture

- All peripherals are mapped into memory address space
- GPIO, timers, ADC, UART accessed using addresses
- No separate I/O instructions
- Programming peripherals = reading/writing registers

Key Insight

Embedded programming is controlled manipulation of memory locations

STM32 Programming Philosophy

- Understand hardware through registers first
- Then move to abstraction layers
- Avoid black-box programming
- Enables:
  - Debugging
  - Optimization
  - Driver development

Programming Approaches in STM32

- Register-level programming
- CMSIS (Core support)
- HAL drivers
- Low Layer (LL) drivers

Course Strategy

Start from registers and gradually move upward

Development Tools Used

- STM32CubeIDE
- ARM GCC toolchain
- ST-Link debugger
- Datasheet and Reference Manual

What You Will Learn in Unit II

- GPIO driver development
- UART, SPI, and I2C communication
- ADC interfacing
- Timers and SysTick
- Interrupt and DMA programming

Lecture Summary

- ARM dominates modern embedded systems
- STM32 integrates ARM core with peripherals
- Everything is memory-mapped
- Strong fundamentals are essential before driver coding

GPIO Architecture and Programming

Lecture Objectives

After this lecture, students will be able to:

- Understand GPIO hardware architecture in STM32
- Explain GPIO pin modes and configurations
- Identify GPIO registers and their roles
- Program GPIO at register level
- Compare polling and interrupt-based GPIO

Why GPIO is Fundamental

- GPIO stands for General Purpose Input Output
- It is the simplest yet most critical peripheral
- Almost every embedded application starts with GPIO:
  - LED control
  - Button reading
  - Sensor interfacing
- Understanding GPIO builds confidence in hardware-level programming

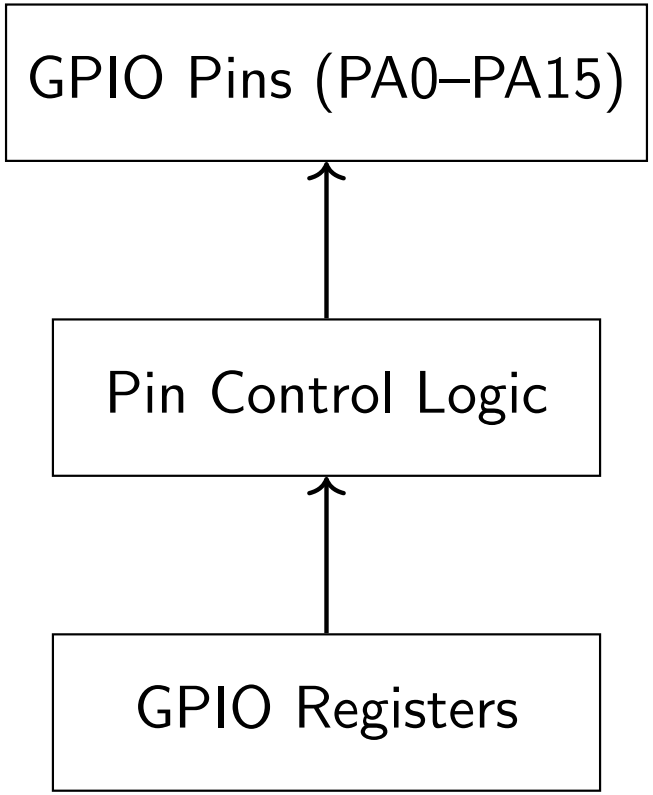
GPIO as an Interface Between Software and Hardware

- GPIO pins act as physical interfaces
- Software writes to registers
- Hardware reacts by changing voltage levels
- GPIO bridges digital logic and real-world signals

STM32 GPIO Port Organization

- GPIO pins are grouped into ports
- Each port is named alphabetically: GPIOA, GPIOB, GPIOC, etc.
- Each port typically has 16 pins:
  - PA0 to PA15
  - PB0 to PB15
- Each port has a dedicated set of registers

GPIO Port Structure



Registers control the behavior of physical pins

GPIO Pin Modes

Each GPIO pin can be configured in one of the following modes:

- Input mode
- Output mode
- Alternate function mode
- Analog mode

Important

Mode selection decides how the pin interacts with internal circuitry

Input Mode

- Pin reads external signal
- Used for buttons, switches, sensors
- Input can be:
  - Floating
  - Pull-up
  - Pull-down
- Pull resistors prevent undefined logic levels

Output Mode

- Pin drives external load
- Used for LEDs, relays, control signals
- Output types:
  - Push-pull
  - Open-drain
- Output speed controls rise and fall time

Alternate Function Mode

- Pin controlled by peripheral instead of GPIO logic
- Used for:
  - UART
  - SPI
  - I2C
  - Timers
- Multiplexing allows one pin to serve multiple functions

Analog Mode

- Disconnects digital input/output circuitry
- Used for ADC and DAC
- Reduces power consumption
- Improves analog accuracy

Key GPIO Registers Overview

Each GPIO port has the following major registers:

- MODER – Mode register
- OTYPER – Output type register
- OSPEEDR – Output speed register
- PUPDR – Pull-up pull-down register
- IDR – Input data register
- ODR – Output data register
- BSRR – Bit set/reset register

GPIO MODER Register

- Controls pin mode
- Each pin uses 2 bits
- Configuration:
  - 00 – Input
  - 01 – Output
  - 10 – Alternate function
  - 11 – Analog

Exam Tip

MODER is always the first register to configure

GPIO Output Registers

- ODR reflects output state
- Writing 1 drives pin high
- Writing 0 drives pin low
- BSRR allows atomic bit set/reset

Why BSRR?

Prevents race conditions during read-modify-write

GPIO Input Register

- IDR holds current pin logic level
- Read-only register
- Used for polling-based input reading

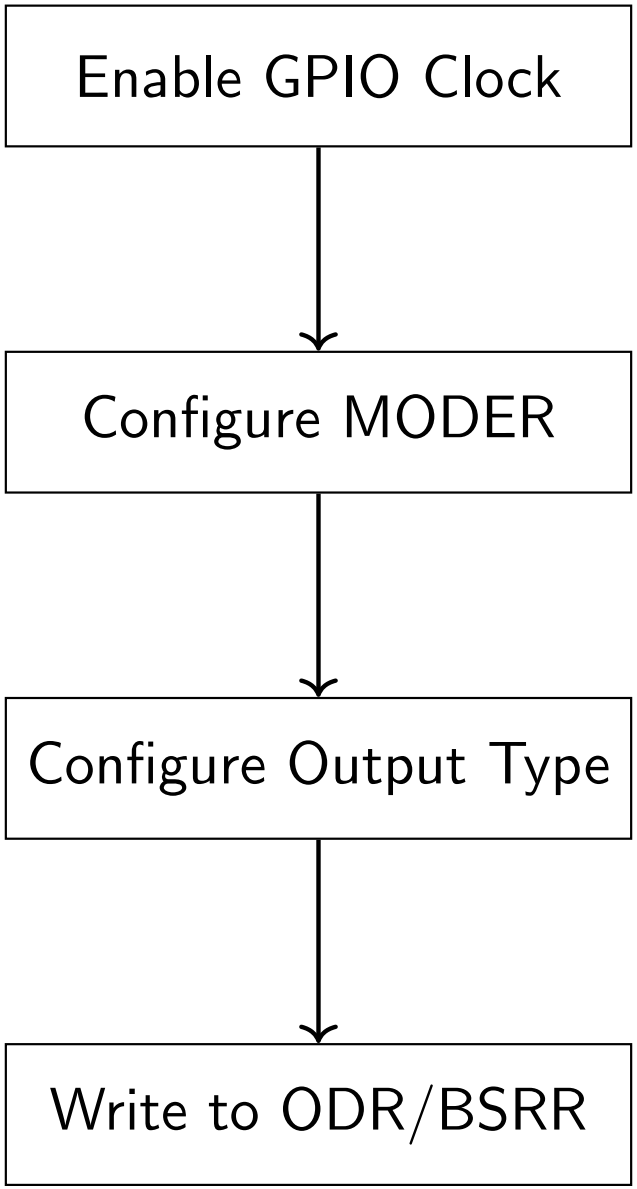
GPIO Register Access Concept

- Registers are memory-mapped
- Accessed using pointers in C
- Writing to address modifies hardware

Core Concept

Software does not touch pins directly, it touches registers

Basic GPIO Output Flow



GPIO Polling vs Interrupt

- Polling:
  - CPU continuously checks pin
  - Simple but inefficient
- Interrupt:
  - Hardware notifies CPU
  - Efficient and scalable

Note

Interrupt-based GPIO will be covered in a later lecture

Common GPIO Programming Mistakes

- Forgetting to enable GPIO clock
- Incorrect MODER bit calculation
- Using ODR instead of BSRR
- Floating input pins

GPIO in Real Applications

- LED blinking
- Button debouncing
- Sensor trigger signals
- Control of external modules

Lecture Summary

- GPIO is the foundation of embedded systems
- STM32 GPIO is register-controlled
- MODER defines pin behavior
- Correct sequencing is essential

GPIO Programming: Register-Level Coding and Examples

Lecture Objectives

After this lecture, students will be able to:

- Write register-level GPIO programs in C
- Enable GPIO peripheral clocks
- Configure GPIO pins as input and output
- Control LEDs using GPIO
- Read button input using polling
- Understand bit manipulation in embedded C

Prerequisite Knowledge

Before coding GPIO, students must understand:

- Memory-mapped I/O
- GPIO register set
- Bitwise operators in C
- STM32 reference manual structure

Register-Level Programming Philosophy

- No HAL (Hardware Abstruction Layer)
- No library abstraction
- Direct interaction with hardware registers
- Full control and transparency

Key Advantage

You know exactly what the hardware is doing

Step 1: Understanding Base Addresses

- Peripheral registers have fixed base addresses
- Example for STM32:
  - Reset and Clock Control (RCC) base address
  - GPIOA base address
- Offsets are added to base address to access registers

Important GPIO Registers Used

- RCC AHB1ENR
- GPIOx MODER
- GPIOx OTYPER
- GPIOx OSPEEDR
- GPIOx PUPDR
- GPIOx ODR
- GPIOx IDR
- GPIOx BSRR

Step 2: Enabling GPIO Clock

- GPIO will not work unless its clock is enabled
- RCC controls clock distribution
- Each GPIO port has a corresponding enable bit

Concept

No clock means no register access

Clock Enable Code Example

```
#define RCC_AHB1ENR  (*(volatile unsigned int*)0x40023830)

void gpio_clock_enable(void)
{
    RCC_AHB1ENR |= (1 << 0);   // Enable GPIOA clock
}
```

Why volatile Keyword is Mandatory

- Prevents compiler optimization
- Register value can change due to hardware
- Ensures actual memory access

Exam Point

Missing volatile causes unpredictable behavior

Step 3: GPIO Pin Mode Configuration

- MODER register controls pin mode
- Each pin uses 2 bits
- Example:
  - 00 input
  - 01 output

Configuring PA5 as Output

```
#define GPIOA_MODER (*(volatile unsigned int*)0x40020000)

void gpio_output_config(void)
{
    GPIOA_MODER &= ~(3 << (5 * 2));
    GPIOA_MODER |=  (1 << (5 * 2));
}
```

Explanation of MODER Code

- Clear both bits for pin 5
- Set bit pattern to 01
- Pin becomes general-purpose output

Step 4: Writing Output Data

- Use ODR or BSRR
- BSRR is preferred
- Atomic operation

LED ON and OFF Using BSRR

```
#define GPIOA_BSRR (*(volatile unsigned int*)0x40020018)

void led_on(void)
{
    GPIOA_BSRR = (1 << 5);
}

void led_off(void)
{
    GPIOA_BSRR = (1 << (5 + 16));
}
```

Why Use BSRR Instead of ODR

- Avoids read-modify-write issue
- Safe in interrupt-driven systems
- Faster execution

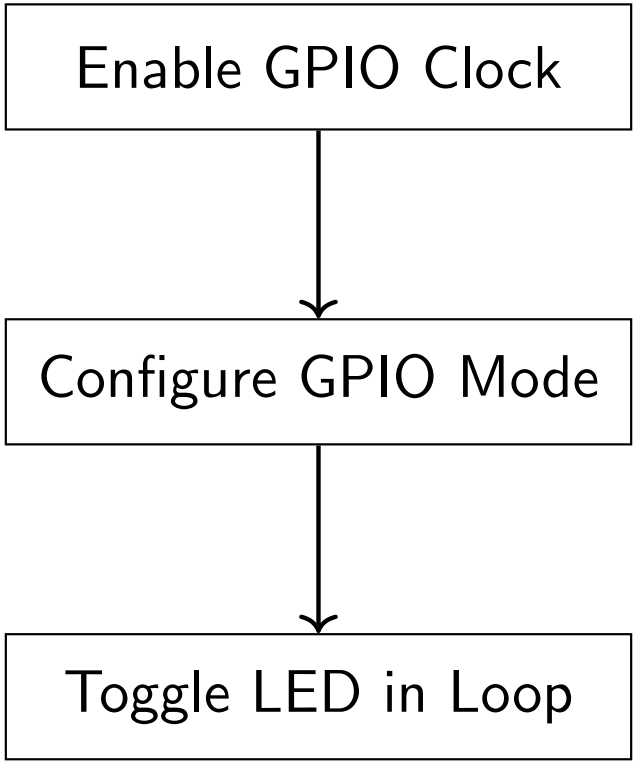
Step 5: Creating Delay

- Simple software delay loop
- Not accurate but useful for learning

Simple Delay Function

```
void delay(volatile unsigned int count)
{
    while(count--);
}
```

Complete LED Blinking Program Flow



Step 6: GPIO Input Reading

- Configure pin as input
- Read IDR register
- Polling-based input

Button Input Example

```
#define GPIOA_IDR (*(volatile unsigned int*)0x40020010)

int read_button(void)
{
    return (GPIOA_IDR & (1 << 0)) ? 1 : 0;
}
```

Polling Logic Explanation

- CPU continuously checks input
- Simple but inefficient
- Suitable for basic applications

Common GPIO Coding Errors

- Clock not enabled
- Wrong bit shift calculation
- Missing volatile
- Floating input pins

Exam-Oriented Questions

- Why is BSRR preferred over ODR?
- Explain role of MODER register
- What happens if GPIO clock is not enabled?
- Explain use of volatile keyword

Lecture Summary

- GPIO can be fully controlled using registers
- Clock enable is mandatory
- Bit manipulation is the heart of embedded C
- This approach builds strong debugging skills

Next Lecture

USART Protocol and Serial Communication Basics